15-288 Ml in a Nutshell

Mohamed Waiel Shikfa

First Edition

Disclaimer

This document aims to summarize the content of the slides and notebooks of 15-288, including what the author considers important. As always, the definition of important is highly subjective so the author might have omitted something that was important to another person, or included something that is trivial to another.

Note: The image chapter contain all the covered content but the order is a bit different than the one in the notebook (will fix it when i have time).

Good luck,

MWS

Feature Engineering for Images (Part 1)

Introduction to Image Data

Before feeding images into traditional machine learning models (like SVMs, Random Forests, or Logistic Regression), we must transform raw pixel data into meaningful, structured representations. This process is known as **Feature Engineering**.

Definition 1 (Image Representation)**.** Mathematically, an image $\mathbf{I}$ is represented as a tensor:

$$\mathbf{I} \in \mathbb{R}^{H \times W \times C}$$

where H is the height, W is the width, and C is the number of color channels.

But in reality, a digital image is fundamentally a discrete, multidimensional grid of numerical values.

Grayscale Images: $C = 1$. Each pixel represents intensity, typically ranging from 0 (black) to 255 (white) in an 8-bit integer format.

Color Images (RGB): $C = 3$. Each pixel is a vector of three values corresponding to Red, Green, and Blue intensities.

Remark 1. Raw pixel values alone are highly sensitive to variations in lighting, translation, rotation, and scaling. Therefore, passing raw, flattened pixels directly to a model (where $\mathbf{x} \in \mathbb{R}^{H \cdot W \cdot C}$) usually yields poor generalization. We must extract invariant and robust features.

Image Metadata

Before processing pixel intensities, many image formats contain embedded **Metadata** (e.g., EXIF data). While not part of the visual signal itself, these attributes provide critical context for feature engineering:

- **Geometric Metadata:** Image height (H), width (W), and aspect ratio.
- **Photometric Metadata:** Color space (RGB, Grayscale, HSV), bit depth (e.g., 8-bit vs. 16-bit), and compression type.
- **Acquisition Data:** Camera settings (shutter speed, aperture, ISO), timestamps, and GPS coordinates which can help normalize environmental variations.

Color Spaces and Transformations

The default RGB color space is heavily correlated with illumination. Transforming the color space is often the first step in feature engineering to decouple color information from lighting.

- **Grayscale Conversion:** Reduces dimensionality and removes color variance if color is irrelevant to the task. The standard luminosity formula accounts for human perception:

$$Y = 0.299R + 0.587G + 0.114B$$

- **HSV / HSL (Hue, Saturation, Value/Lightness):**

  Hue: Represents the pure color type (e.g., "red", "blue") as an angle from 0° to 360°.

  Saturation: Represents the intensity or purity of the color.

  Value: Represents the brightness.

  By isolating the *Value* channel, we can achieve invariance to shadows and lighting conditions while retaining the *Hue* for color-based classification.

Basic Image Features

1. Global Features: Color Histograms

A color histogram describes the global distribution of colors in an image, completely ignoring spatial layout.

Mechanism: The color space is divided into a set of discrete "bins." We iterate through all pixels in the image and increment the bin corresponding to the pixel's color value.

Normalization: To ensure the feature vector is invariant to image size, the histogram is normalized (divided by the total number of pixels), producing a discrete probability distribution.

Advantages: Perfectly invariant to translation and rotation. Highly robust to scaling and partial occlusion.

Disadvantages: Complete loss of spatial information. An image of a red apple on green grass will have the exact same histogram as an image of scattered red and green pixels.

2. Local Features: Spatial Patching

To retain spatial context while avoiding the rigidity of full-image flattening, we can divide the image into a grid of sub-regions (patches) and compute basic features (like histograms or mean intensities) for each patch independently.

Spatial Filtering and Convolution

To extract structural features (edges, corners, textures), we rely on local neighborhood operations. This involves sliding a small matrix across the image.

Definition 2 (Kernel / Filter)**.** A kernel $\mathbf{K}$ is a small, typically square matrix (e.g., 3×3 or 5×5) of weights. It defines how a pixel's neighborhood influences its new value.

Aggregate Statistics (Global Features)

A fundamental approach to reducing the dimensionality of an image $\mathbf{I}$ is to treat pixel intensities as a distribution and calculate its moments. These statistics provide a global summary of the image's brightness and contrast.

Mean (μ): Represents the average brightness of the image.

$$\mu = \frac{1}{H \cdot W} \sum_{i=1}^{H} \sum_{j=1}^{W} I(i,j)$$

Variance (σ^2): Measures the "spread" of intensities, often correlating with global image contrast.

$$\sigma^2 = \frac{1}{H \cdot W} \sum_{i=1}^{H} \sum_{j=1}^{W} (I(i,j) - \mu)^2$$

Skewness: Measures the asymmetry of the intensity distribution around the mean. A positive skew indicates a distribution with a tail toward higher (brighter) intensities.

Kurtosis: Describes the "peakedness" or the presence of outliers in the intensity distribution. High kurtosis suggests the image contains many pixels that are much brighter or darker than the average (high contrast/texture).

Remark 2. While these aggregate statistics are computationally efficient and invariant to rotation or translation, they discard all spatial information. Consequently, they are typically used as auxiliary features alongside local descriptors like SIFT or HOG.

Cross-Correlation vs. Convolution

Let $\mathbf{I}$ be the image and $\mathbf{K}$ be a kernel of size $(2k+1) \times (2k+1)$. The new value of a pixel at position (i,j) is computed as:

Cross-Correlation: The kernel is laid directly over the image.

$$G[i,j] = (\mathbf{I} \otimes \mathbf{K})[i,j] = \sum_{u=-k}^{k} \sum_{v=-k}^{k} \mathbf{K}[u,v]\mathbf{I}[i+u, j+v]$$

Convolution: The kernel is vertically and horizontally flipped before multiplication. In signal processing, this ensures the operation is associative. In machine learning, the terms are often used interchangeably because weights are learned.

$$G[i,j] = (\mathbf{I} * \mathbf{K})[i,j] = \sum_{u=-k}^{k} \sum_{v=-k}^{k} \mathbf{K}[u,v]\mathbf{I}[i-u, j-v]$$

Padding and Stride

- **Padding:** Applying a filter reduces the spatial dimensions of the image. To preserve dimensions, we add artificial pixels around the border. Common strategies include *Zero-padding* (adding zeros), *Replicate* (copying edge pixels), and *Reflect* (mirroring).
- **Stride:** The step size with which the filter slides across the image. A stride > 1 results in spatial downsampling.

Common Hand-Crafted Kernels

Filter Type	**Example 3×3 Kernel**	**Purpose and Effect**
Box Blur	$\frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$	Averages the neighborhood. Smooths the image and acts as a low-pass filter, removing high-frequency noise but blurring edges.
Gaussian Blur	$\frac{1}{16}\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$	Weights central pixels more heavily than distant ones. Provides a more natural, rotationally symmetric smoothing compared to a box blur.
Sharpen	$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$	Enhances differences between a pixel and its neighbors. Acts as a high-pass filter, emphasizing edges and fine details.

Edge Detection and Gradients

Edges define the boundaries of objects. An edge is essentially a region in the image where there is a rapid change in pixel intensity. We can detect edges by computing the discrete derivative of the image.

Image Gradients

The gradient of an image $\mathbf{I}(x,y)$ is a vector containing the first-order partial derivatives:

$$\nabla \mathbf{I} = \begin{bmatrix} \frac{\partial \mathbf{I}}{\partial x} \\ \frac{\partial \mathbf{I}}{\partial y} \end{bmatrix}$$

Since images are discrete, we approximate derivatives using finite differences (which can be implemented as convolution kernels).

Sobel Operators

The Sobel filters compute the gradient while applying a slight Gaussian smoothing to reduce noise.

$$\mathbf{K}_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} \quad \text{(Detects vertical edges)}$$

$$\mathbf{K}_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix} \quad \text{(Detects horizontal edges)}$$

Once we compute $G_x = \mathbf{I} * \mathbf{K}_x$ and $G_y = \mathbf{I} * \mathbf{K}_y$, we can derive two crucial features for every pixel:

Gradient Magnitude: The strength of the edge.

$$M = \sqrt{G_x^2 + G_y^2}$$

Gradient Direction: The orientation of the edge (perpendicular to the edge itself).

$$\theta = \arctan\left(\frac{G_y}{G_x}\right)$$

Remark 3. Gradient magnitudes and orientations form the foundation for advanced feature descriptors such as **HOG** (Histogram of Oriented Gradients) and **SIFT** (Scale-Invariant Feature Transform), which represent the state-of-the-art in classical Computer Vision feature engineering.

Canny Edge Detection Pipeline

A sophisticated, multi-stage algorithm for finding optimal edges:

1. **Noise Reduction:** Apply a Gaussian blur to smooth the image.
2. **Gradient Calculation:** Use Sobel operators to find M and θ.
3. **Non-Maximum Suppression:** Thin the edges by suppressing any pixel that is not a local maximum in the direction of the gradient θ.
4. **Hysteresis Thresholding:** Use two thresholds (T_{high} and T_{low}). Pixels $> T_{high}$ are strong edges. Pixels between thresholds are weak edges and only kept if connected to a strong edge.

Feature Engineering for Images (Part 2)

Image Binarization and Thresholding

Before extracting complex structures, it is often useful to separate the foreground (objects of interest) from the background. We achieve this by converting a grayscale image into a binary image (0s and 1s).

1. Global Thresholding

A single, hard cutoff value T is applied to the entire image.

$$\text{Pixel}_{new} = \begin{cases} 255 & \text{if Pixel}_{old} > T \\ 0 & \text{otherwise} \end{cases}$$

Remark 4. Global thresholding fails miserably if the image has a lighting gradient (e.g., uneven shadows across a document), as one side will turn entirely black and the other entirely white.

2. Adaptive (Local) Thresholding

Instead of one global T, a local threshold is calculated independently for small regions (e.g., an 11×11 pixel block) of the image.

- **Mean Thresholding:** The threshold is the mean of the neighborhood area.
- **Gaussian Thresholding:** The threshold is the weighted sum of neighborhood values, where weights are a Gaussian window.

This naturally handles varying illumination across the image.

3. Otsu's Binarization

Otsu's method is an algorithm that automatically discovers the optimal global threshold T. It assumes the image contains two classes of pixels (foreground and background) forming a bimodal histogram. It iterates through all possible threshold values to find the one that minimizes *intra-class variance* (or equivalently, maximizes *inter-class variance*).

Local Operations & Corner Detection

To track objects across different images, we must find distinct, repeatable points.

- **Flat Regions:** No gradient in any direction. Useless for tracking.
- **Edges:** Gradient in one direction. Good, but suffers from the "aperture problem" (we cannot determine where exactly we are along the edge).
- **Corners:** High gradients in *all* directions. Highly distinct and excellent for tracking.

Harris Corner Detector

The Harris Corner Detector formally identifies corners by analyzing the change in intensity resulting from shifting a local window by a small amount (u, v).

The core of the algorithm relies on the **Structure Tensor** (or Second Moment Matrix), denoted M, which is computed from the image derivatives I_x and I_y over a local window W:

$$M = \sum_{(x,y)\in W} \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix}$$

By analyzing the eigenvalues (λ_1, λ_2) of this matrix, we categorize the region:

Corner: Both λ_1 and λ_2 are large. Shifting the window in any direction yields a large change.

Edge: One eigenvalue is large, the other is small.

Flat: Both eigenvalues are small.

The Feature Matching Pipeline

Once we find distinct points, how do we match them between two different images (e.g., for panorama stitching or 3D reconstruction)? The standard classical pipeline consists of three steps:

1. **Detection (Keypoints):** Identify repeatable interest points (like Harris corners or blobs) in the image.
2. **Description (Descriptors):** Extract a vector that mathematically describes the neighborhood around the keypoint. This descriptor must be robust to lighting and rotation.
3. **Matching:** Compute the distance (e.g., Euclidean or Hamming distance) between descriptors in Image A and Image B to find corresponding pairs.

Advanced Feature Descriptors

While Harris corners are invariant to rotation, they are **not scale-invariant** (if you zoom in closely on a corner, it looks like an edge!). State-of-the-art classical computer vision utilizes descriptors that overcome this.

SIFT (Scale-Invariant Feature Transform)

SIFT is arguably the most famous feature descriptor. It is highly robust but computationally heavy.

Detection: Uses the *Difference of Gaussians (DoG)* at multiple scales to find blobs (keypoints) that exist consistently across different zoom levels.

Orientation Assignment: Computes the dominant gradient direction around the keypoint and aligns the descriptor to it, ensuring true rotation invariance.

Description: Creates a 4×4 grid around the keypoint. In each of the 16 cells, it computes a Histogram of Oriented Gradients (HOG) with 8 bins. This results in a highly descriptive $16 \times 8 = 128$-dimensional continuous feature vector.

ORB (Oriented FAST and Rotated BRIEF)

ORB was developed as an efficient, fast, and free alternative to SIFT, making it heavily utilized in real-time robotics and mobile applications.

FAST Detection: Rapidly finds keypoints by comparing the intensity of a central pixel to a circular ring of 16 surrounding pixels. If contiguous pixels are significantly brighter or darker, it's a corner.

BRIEF Description: Instead of computing continuous gradients, BRIEF creates a *binary descriptor* (a string of 0s and 1s) by doing simple pairwise intensity comparisons between random pixels in the neighborhood.

Matching: Because descriptors are binary, matching is extremely fast using the Hamming distance (computed via a fast XOR operation on the CPU).

Feature Engineering for Images (Part 3)

Combining Filters: Unsharp Masking

Filters are linear operations, which means they possess the property of superposition and can be algebraically combined. A classic example is the **Sharpening Filter**, which can be derived from a combination of the original image and a smoothing filter.

This process is commonly known as **Unsharp Masking**. It can be broken down into two logical steps:

1. **Extract Details (High-Pass):** Take a smoothed (blurred) version of the image and subtract it from the original image. This removes the low-frequency information (smooth gradients, general color) and leaves only the high-frequency *details* (edges and noise).

$$I_{details} = I_{original} - I_{smoothed}$$

2. **Enhance Image:** Add these extracted details back to the original image, scaled by a factor α, to emphasize the edges.

$$I_{sharp} = I_{original} + \alpha \cdot I_{details}$$

Linear Models: Definitions, Properties, and Training

Introduction to Linear Models

Linear models operate under the hypothesis that the target output can be well-described by a weighted linear combination of the input features. Despite their simplicity, they are fundamental, interpretable, and serve as the building blocks for more complex algorithms like neural networks.

Definition 3 (Linear Model)**.** For an input vector $\mathbf{x} = [x_1, x_2, \ldots, x_d]^T$ with d features, the linear model outputs a prediction $f_{\mathbf{w}}(\mathbf{x})$ computed as:

$$f_{\mathbf{w}}(\mathbf{x}) = w_1 x_1 + w_2 x_2 + \cdots + w_d x_d + w_0 = \mathbf{w}^T \mathbf{x} + w_0$$

where $\mathbf{w} = [w_1, \ldots, w_d]^T$ is the weight vector and w_0 is the bias (or intercept) term.

Remark 5 (Extended Feature Vector)**.** To simplify notation and avoid treating the bias w_0 separately, we typically augment the input vector with a constant 1. Let $\mathbf{x}' = [1, x_1, \ldots, x_d]^T$ and $\mathbf{w}' = [w_0, w_1, \ldots, w_d]^T$. The prediction becomes a simple dot product:

$$f_{\mathbf{w}}(\mathbf{x}) = (\mathbf{w}')^T \mathbf{x}'$$

Classification vs. Regression

The interpretation of the linear output $f_{\mathbf{w}}(\mathbf{x})$ depends on the task:

Regression: The target output is a continuous curve fitting the data. The prediction is simply the raw output of the linear model: $\hat{y} = f_{\mathbf{w}}(\mathbf{x})$.

Classification: The target output is a linear decision boundary (a hyperplane) that separates classes. For binary classification ($y \in \{-1, +1\}$), the prediction is the sign of the model's output: $\hat{y} = \text{sgn}(f_{\mathbf{w}}(\mathbf{x}))$.

Definition 4 (Functional Margin)**.** In binary classification, the **functional margin** is defined as $y \cdot f_{\mathbf{w}}(\mathbf{x})$, where $y \in \{-1, +1\}$ is the true label.

- If $y \cdot f_{\mathbf{w}}(\mathbf{x}) > 0$, the prediction is **correct** (the sign of the prediction matches the true label).
- If $y \cdot f_{\mathbf{w}}(\mathbf{x}) < 0$, the prediction is **incorrect**.

The magnitude of the margin $|f_{\mathbf{w}}(\mathbf{x})|$ represents the *confidence* of the prediction.

Non-Linear Feature Transformations

What if the data is not linearly separable or linearly distributed? We can still use linear models by applying a **feature transformation**.

Remark 6. We can map the original feature vector $\mathbf{x}$ into a higher-dimensional space using a non-linear function $\phi(\mathbf{x})$. For example, if $\mathbf{x} = [x_1, x_2]^T$, we could transform it into polynomial features:

$$\phi(\mathbf{x}) = [1, x_1, x_2, x_1^2, x_2^2, x_1 x_2]^T$$

The model $f_{\mathbf{w}}(\mathbf{x}) = \mathbf{w}^T \phi(\mathbf{x})$ is **linear in the transformed parameter space**, but represents a **non-linear boundary/curve in the original input space**.

Training: Empirical Risk Minimization

Training a linear model means finding the optimal weights $\mathbf{w}^*$ that minimize the error over the training dataset. This is formalized as Empirical Risk Minimization (ERM).

Definition 5 (Empirical Risk Minimization)**.** The goal is to find $\mathbf{w}$ that minimizes the average loss over the N training examples:

$$\mathbf{w}^* = \arg\min_{\mathbf{w}} \frac{1}{N} \sum_{i=1}^{N} \ell\left(f_{\mathbf{w}}(\mathbf{x}^{(i)}), y^{(i)}\right)$$

where $\ell(\hat{y}, y)$ is the **loss function** measuring the penalty for a bad prediction.

Common Loss Functions

For Regression ($y \in \mathbb{R}$):

Squared Loss (L_2 Loss): $\ell(\hat{y}, y) = (y - \hat{y})^2$. Strongly penalizes large outliers. Used in Ordinary Least Squares (OLS).

Absolute Loss (L_1 Loss): $\ell(\hat{y}, y) = |y - \hat{y}|$. More robust to outliers.

For Classification ($y \in \{-1, +1\}$, margin $\mu = y f_{\mathbf{w}}(\mathbf{x})$):

0-1 Loss: $\ell(\mu) = 1$ if $\mu < 0$, else 0. Ideal but computationally intractable (non-differentiable step function).

Hinge Loss: $\ell(\mu) = \max(0, 1 - \mu)$. Used in Support Vector Machines (SVM).

Logistic Loss: $\ell(\mu) = \log(1 + e^{-\mu})$. Smooth, differentiable convex surrogate. Used in Logistic Regression.

Exponential Loss: $\ell(\mu) = e^{-\mu}$. Highly sensitive to misclassifications. Used in AdaBoost.

Regularization

To prevent overfitting, especially when working with high-dimensional feature spaces (like expanded polynomials), we add a **regularization penalty** $R(\mathbf{w})$ to the loss function. The new objective becomes:

$$\min_{\mathbf{w}} \left(L_{\text{emp}}(\mathbf{w}) + \lambda R(\mathbf{w})\right)$$

where $\lambda > 0$ controls the strength of the penalty.

$\mathbf{L_2}$ Regularization (Ridge): $R(\mathbf{w}) = ||\mathbf{w}||_2^2 = \sum w_j^2$. Penalizes large weights, driving them closer to zero, which shrinks variance and creates smoother decision functions.

$\mathbf{L_1}$ Regularization (Lasso): $R(\mathbf{w}) = ||\mathbf{w}||_1 = \sum |w_j|$. Induces **sparsity**, meaning it forces less important feature weights exactly to zero, performing intrinsic feature selection.

Optimization: Finding the Optimal Weights

How do we actually solve the arg min problem?

1. **Analytical Solution (Closed-form):** Set the gradient to zero ($\nabla_{\mathbf{w}} L(\mathbf{w}) = \mathbf{0}$) and solve for $\mathbf{w}$. This is possible for some models like unregularized or L_2-regularized linear regression.
2. **Numerical Optimization (Gradient Descent):** For loss functions without closed-form solutions (like Logistic Loss), we use iterative methods. We compute the gradient $\nabla_{\mathbf{w}} L(\mathbf{w})$ and take steps in the opposite direction.

Definition 6 (Gradient Descent Update Rule)**.** At each iteration t, the weights are updated using the rule:

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \nabla_{\mathbf{w}} L(\mathbf{w}^{(t)})$$

where η is the **learning rate** (step size), determining how far we move along the slope of the error surface at each step.

Gradient Methods for Model Optimization

The Optimization Problem

Once a model hypothesis $f_{\mathbf{w}}(\mathbf{x})$ and a loss function $\ell(\hat{y}, y)$ are defined, the machine learning task reduces to an optimization problem: Empirical Risk Minimization (ERM). Our goal is to find the optimal parameter vector $\mathbf{w}^*$ that minimizes the average loss over the training set:

$$\mathbf{w}^* = \arg\min_{\mathbf{w}} L(\mathbf{w}) = \arg\min_{\mathbf{w}} \frac{1}{N} \sum_{i=1}^{N} \ell\left(f_{\mathbf{w}}(\mathbf{x}^{(i)}), y^{(i)}\right)$$

While some linear models (like Ordinary Least Squares) have a closed-form analytical solution by setting $\nabla_{\mathbf{w}} L(\mathbf{w}) = \mathbf{0}$, most complex models (such as logistic regression and neural networks) result in loss functions that cannot be solved analytically. Thus, we must rely on iterative numerical optimization.

Gradient Descent Fundamentals

Definition 7 (Gradient Descent (GD))**.** Gradient Descent is an iterative optimization algorithm for finding a local minimum of a differentiable function. At each step, it calculates the gradient $\nabla_{\mathbf{w}} L(\mathbf{w})$ (the vector of partial derivatives) at the current point $\mathbf{w}^{(t)}$ and takes a step in the exact opposite direction, as the negative gradient points towards the steepest descent.

The fundamental update rule at iteration t is:

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \nabla_{\mathbf{w}} L(\mathbf{w}^{(t)})$$

The Learning Rate (η): Also called the step size, η determines how far the algorithm moves along the negative gradient.

- If η is **too small**, convergence will be painstakingly slow, risking getting trapped in sub-optimal regions.
- If η is **too large**, the updates can overshoot the minimum, leading to oscillation or complete divergence.

Remark 7 (Convexity vs. Non-Convexity)**.** If the loss surface $L(\mathbf{w})$ is **convex** (bowl-shaped, like in Linear Regression or Logistic Regression), GD is guaranteed to eventually converge to the single global minimum. If the surface is **non-convex** (like in deep neural networks), the algorithm may get stuck in *local minima*, *saddle points*, or flat *plateaus*.

Data-Sampling Variants of Gradient Descent

Computing the gradient requires summing the loss derivatives over the training data. Depending on how much data we use per step, we define three main variants of GD:

Batch Gradient Descent (BGD): Computes the gradient using the **entire** training dataset (N samples) for a single update.

- *Pros:* Provides a highly accurate estimate of the gradient; guarantees stable convergence.
- *Cons:* Computationally expensive and memory-intensive. Impractical for massive datasets as one tiny step requires a full pass over all data.

Stochastic Gradient Descent (SGD): Computes the gradient using only **one randomly selected** training example per update.

- *Pros:* Extremely fast per iteration; low memory requirement. The inherent noise from single samples can help the optimizer "bounce" out of shallow local minima.
- *Cons:* High variance in updates causes the loss to fluctuate wildly, preventing it from settling smoothly at the exact minimum.

Mini-Batch Gradient Descent: The standard compromise. It computes the gradient on a small, random **subset** (a mini-batch) of B training examples (e.g., $B = 32, 64, 128$).

- *Pros:* Significantly reduces the variance of the parameter updates compared to pure SGD, leading to more stable convergence. Crucially, it exploits hardware optimizations (like GPU matrix vectorization) making it much faster than BGD.

Advanced Gradient Methods

Standard SGD/Mini-batch GD can struggle with pathological curvature (e.g., steep ravines) and can be slow. Advanced optimizers manipulate the update rule to accelerate and stabilize convergence.

Momentum-Based Methods

Definition 8 (Momentum)**.** Instead of depending strictly on the current gradient, Momentum accumulates an exponentially decaying moving average of past gradients to determine the update direction. It acts like a ball rolling down a hill, gaining speed in consistent directions and dampening oscillations in erratic directions.

Standard Momentum: Computes a "velocity" vector $v_t = \gamma v_{t-1} + \eta \nabla_{\mathbf{w}} L(\mathbf{w})$. The weights are updated as $\mathbf{w} \leftarrow \mathbf{w} - v_t$.

Nesterov Accelerated Gradient (NAG): A "look-ahead" version of momentum. Instead of evaluating the gradient at the current position $\mathbf{w}$, it calculates the gradient at the *predicted* future position $\mathbf{w} - \gamma v_{t-1}$. This anticipatory step prevents the algorithm from overshooting the minimum.

Adaptive Learning Rate Methods

Instead of using a single global learning rate η for all parameters, adaptive methods dynamically tune the learning rate for each individual parameter w_j.

AdaGrad (Adaptive Gradient): Scales the learning rate for each parameter inversely proportional to the square root of the sum of all its historical squared gradients. It performs larger updates for infrequent features and smaller updates for frequent features. *Drawback:* The accumulated sum continuously grows, eventually causing the learning rate to shrink to zero, stopping training prematurely.

RMSprop: Fixes AdaGrad's diminishing learning rate by using an exponentially decaying average of squared gradients, rather than a cumulative sum.

Adam (Adaptive Moment Estimation)

Definition 9 (Adam Optimizer)**.** Adam is currently the most popular default optimizer in deep learning. It combines the benefits of both **Momentum** (smoothing via the 1st statistical moment) and **RMSprop** (adaptive scaling via the 2nd statistical moment).

The Adam update rule consists of 4 formal steps at iteration t:

1. **Momentum (1st Moment):** Compute the exponentially weighted average of past gradients.

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) \nabla f(\mathbf{w}_{t-1})$$

   This smooths updates and reduces oscillations.

2. **Adaptive Scaling (2nd Moment):** Compute the exponentially weighted average of past *squared* gradients.

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2)(\nabla f(\mathbf{w}_{t-1}))^2$$

   This prevents overly large updates when gradients are huge, and ensures reasonable updates in noisy areas.

3. **Bias Correction:** Because m_t and v_t are initialized to zero, they are biased toward zero early in training. We correct them:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t} \quad \text{and} \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

4. **Parameter Update Rule:**

$$\mathbf{w}_t = \mathbf{w}_{t-1} - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon}\hat{m}_t$$

(Where ϵ is a tiny constant to prevent division by zero).

Remark 8 (Why Adam Works)**.** The effective learning rate in Adam is heavily adaptive. If the gradient magnitude is consistently large, the step size safely decreases. If gradients are small (e.g., in a plateau), the step size increases to prevent stalling. Combined with Momentum's directional consistency, Adam achieves rapid and robust convergence in non-convex landscapes.

Support Vector Machines (SVM)

The Optimal Decision Boundary

When training a linear classifier, there may be infinitely many hyperplanes that perfectly separate two linearly separable classes. Support Vector Machines (SVMs) are designed to answer the fundamental question: *Which of these boundaries is the best?*

Intuitively, the best decision boundary is the one that is furthest away from the closest training data points of any class. This concept is formalized using the geometric margin.

Definition 10 (Geometric Margin)**.** The **margin** is defined as the perpendicular distance between the decision boundary (the hyperplane) and the closest data points from each class. SVM seeks to find the hyperplane that **maximizes this margin**.

Definition 11 (Support Vectors)**.** The **support vectors** are the specific data points that lie exactly on the margin boundaries. They are the most difficult points to classify. The entire position and orientation of the optimal hyperplane depend *only* on these support vectors; if all other data points were moved or removed, the decision boundary would not change.

Hard Margin vs. Soft Margin

Hard Margin SVM

A Hard Margin SVM enforces a strict rule: all data points must be correctly classified, and no points are allowed to fall inside the margin.

Mathematically, the total width of the margin is $\frac{2}{||\mathbf{w}||}$. To maximize the margin, we must minimize $||\mathbf{w}||$. The optimization problem is:

$$\min_{\mathbf{w}, w_0} \frac{1}{2}||\mathbf{w}||^2$$

Subject to the strict constraints: $y^{(i)}(\mathbf{w}^T\mathbf{x}^{(i)} + w_0) \geq 1 \quad \text{for all } i.$

Remark 9. Hard Margin SVMs only work if the data is **perfectly linearly separable**. Even a single outlier can drastically skew the boundary or make finding a solution impossible.

Soft Margin SVM

To handle data that is not perfectly separable or contains noisy outliers, we introduce the **Soft Margin SVM**. It allows some points to violate the margin boundary (or even be misclassified) by introducing **slack variables**, denoted by $\xi_i \geq 0$.

The modified optimization problem becomes:

$$\min_{\mathbf{w}, w_0, \xi} \left(\frac{1}{2}||\mathbf{w}||^2 + C\sum_{i=1}^{N} \xi_i \right)$$

The Regularization Parameter (C): The hyperparameter C controls the trade-off between maximizing the margin width and minimizing the classification errors (slack).

- **Large C:** Imposes a high penalty for errors. The model will choose a narrower margin to perfectly classify as many training points as possible (prone to overfitting).
- **Small C:** Imposes a low penalty for errors. The model favors a wider, smoother margin, even if it misclassifies a few training points (better generalization, prone to underfitting).

SVM and the Hinge Loss

We can also view Soft Margin SVM through the lens of Empirical Risk Minimization (ERM) that we established in previous lectures. Training a Soft Margin SVM is mathematically equivalent to minimizing the **Hinge Loss** combined with L_2 Regularization:

$$L(\mathbf{w}) = \sum_{i=1}^{N} \max(0, 1 - y^{(i)} f_{\mathbf{w}}(\mathbf{x}^{(i)})) + \lambda ||\mathbf{w}||^2$$

Where $f_{\mathbf{w}}(\mathbf{x}) = \mathbf{w}^T\mathbf{x} + w_0$. The Hinge loss penalizes predictions not just when they have the wrong sign, but when they fall inside the margin (i.e., when the functional margin $y^{(i)} f_{\mathbf{w}}(\mathbf{x}^{(i)}) < 1$).

The Kernel Trick

Often, the data is highly non-linear and cannot be separated by a flat hyperplane, even with a soft margin. While we could explicitly map the input features $\mathbf{x}$ to a higher-dimensional space $\phi(\mathbf{x})$ (like polynomial features), doing so is computationally expensive for very high dimensions.

Definition 12 (The Kernel Trick)**.** In the dual mathematical formulation of the SVM, the data points only ever appear as dot products: $\mathbf{x}^{(i)} \cdot \mathbf{x}^{(j)}$. The Kernel Trick replaces this dot product with a **Kernel Function** $K(\mathbf{x}^{(i)}, \mathbf{x}^{(j)})$, which computes the dot product in the higher-dimensional space *without ever explicitly transforming the vectors.*

$$K(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) = \phi(\mathbf{x}^{(i)})^T \phi(\mathbf{x}^{(j)})$$

This allows the SVM to learn highly non-linear decision boundaries efficiently. Common kernels include:

- **Linear Kernel:** Standard dot product (no transformation).
- **Polynomial Kernel:** Maps data to a polynomial feature space.
- **RBF / Gaussian Kernel:** Maps data to an infinite-dimensional space, capable of wrapping completely around clusters of data.

Support Vector Regression (SVR)

The concepts of SVMs can be naturally extended from classification to regression tasks.

In standard regression, we penalize the model for any deviation from the true target value. In **Support Vector Regression (SVR)**, we replace the concept of a "classification margin" with an **ϵ-insensitive tube** that surrounds the regression curve.

ϵ-insensitive error: If a predicted value falls *inside* the tube of width ϵ around the true value, the error is ignored (zero loss). We only pay a penalty (using slack variables ξ) for predictions that fall *outside* of this tube.

Like classification SVMs, SVR relies on support vectors (the points identifying the tube) and can be fully kernelized to fit non-linear regression curves.

Logistic Regression

A Probabilistic View of Machine Learning

Up to this point, we have largely viewed classification through a geometric or functional lens (e.g., finding a hyperplane that separates classes). Logistic Regression introduces a **probabilistic formulation** of learning.

Instead of just predicting a discrete label $y \in \{0, 1\}$ given an input $\mathbf{x}$, we want to learn the **conditional probability density** that the label is y, given $\mathbf{x}$. We want our model to output:

$$P(y \mid \mathbf{x})$$

Generative Models: These models attempt to learn the joint probability $P(\mathbf{x}, y)$ or the conditional distribution of the features given the class, $P(\mathbf{x} \mid y)$. They essentially try to model exactly how the data for each class was generated (e.g., Naive Bayes).

Discriminative Models: These models bypass modeling the underlying data distribution and attempt to learn the conditional probability $P(y \mid \mathbf{x})$ directly. **Logistic Regression is a discriminative model.**

The Logistic Sigmoid Function

A standard linear model calculates a raw score $z = \mathbf{w}^T\mathbf{x} + w_0$, which can range from $-\infty$ to $+\infty$. To interpret this output as a probability, we need to map it to the range $[0, 1]$. We achieve this using the Logistic Sigmoid function.

Definition 13 (Logistic Sigmoid Function)**.** The sigmoid function, denoted as $\sigma(z)$, "squashes" any real-valued number into the range $(0, 1)$:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Key Properties of $\sigma(z)$:

- As $z \to \infty$, $\sigma(z) \to 1$.
- As $z \to -\infty$, $\sigma(z) \to 0$.
- When $z = 0$, $\sigma(0) = 0.5$.
- Symmetry property: $\sigma(-z) = 1 - \sigma(z)$.

The Logistic Regression Model

In binary Logistic Regression (where $y \in \{0, 1\}$), the hypothesis function $f_{\mathbf{w}}(\mathbf{x})$ models the probability that the positive class ($y = 1$) is true:

$$\hat{y} = f_{\mathbf{w}}(\mathbf{x}) = P(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^T\mathbf{x} + w_0) = \frac{1}{1 + e^{-(\mathbf{w}^T\mathbf{x} + w_0)}}$$

Consequently, the probability of the negative class is simply $P(y = 0 \mid \mathbf{x}) = 1 - P(y = 1 \mid \mathbf{x})$.

Remark 10 (The Decision Boundary)**.** To make a final hard classification, we typically apply a threshold of 0.5. We predict $y = 1$ if $P(y = 1 \mid \mathbf{x}) \geq 0.5$. Because $\sigma(z) \geq 0.5$ exactly when $z \geq 0$, our prediction rule simplifies to:

$$\text{Predict 1 if } \mathbf{w}^T\mathbf{x} + w_0 \geq 0$$

Therefore, despite the non-linear sigmoid transformation yielding probabilities, **the underlying decision boundary separating the classes is strictly linear** (a flat hyperplane).

Training: Maximum Likelihood and Log-Loss

Because Logistic Regression outputs probabilities, we do not train it using the standard sum of squared errors. Instead, we use **Maximum Conditional Likelihood Estimation (MCLE)**. We want to find the parameters $\mathbf{w}$ that maximize the probability of observing the labels in our training dataset.

Instead of maximizing the raw likelihood (which leads to numerical underflow due to multiplying many small probabilities), we typically **minimize the negative log-likelihood**, which gives us our cost function.

Definition 14 (Binary Cross-Entropy / Log Loss)**.** For a dataset of N examples, the Logistic Regression cost function $L(\mathbf{w})$ is defined as:

$$L(\mathbf{w}) = -\frac{1}{N}\sum_{i=1}^{N}\left[y^{(i)}\log(\hat{y}^{(i)}) + (1-y^{(i)})\log(1-\hat{y}^{(i)})\right]$$

where $\hat{y}^{(i)} = \sigma(\mathbf{w}^T\mathbf{x}^{(i)} + w_0)$ is the predicted probability.

This loss function heavily penalizes the model if it confidently predicts the wrong class (e.g., predicting $\hat{y} \approx 0$ when the true label $y = 1$ causes the $-\log(\hat{y})$ term to approach infinity).

Optimization and Regularization

Unlike Linear Regression, setting the gradient of the Logistic Regression loss function to zero **does not yield a closed-form analytical solution** due to the non-linear e^{-z} term in the sigmoid.

Optimization: We must use iterative optimization techniques, such as **Gradient Descent** (or Gradient Ascent, if maximizing likelihood directly). Fortunately, the Log-Loss function is strictly **convex**, meaning Gradient Descent is guaranteed to find the global optimal weights $\mathbf{w}^*$.

Regularization: If the training data is perfectly linearly separable, the algorithm will try to push the probabilities to exactly 1.0 and 0.0. To do this, the weights $||\mathbf{w}||$ will grow towards infinity, leading to severe overfitting. To prevent this, we must add L_1 (Lasso) or L_2 (Ridge) regularization penalties to the loss function.